

MagicQ miniQ on Dell Wyse 3040

Ubuntu + Openbox + ChamSys MagicQ as a dedicated show PC for the **Compact Wing**, with automatic USB stick mounting.

Note: In many configurations the Wyse 3040 has only **2 GB RAM** and **8 GB eMMC**. MagicQ is relatively resource-hungry and the Compact Wing is required for "unlocked" operation. With 2 GB RAM everything runs tight but works. It is recommended to run **only a lightweight system** (server install + Openbox) and nothing else.

Table of contents

- [1. Requirements / Hardware](#)
- [2. Prepare the BIOS](#)
- [3. Install Ubuntu \(recommended: 24.04 LTS Server\)](#)
- [4. Repair the boot path \(BOOTX64.EFI\) – absolutely required!](#)
- [5. Run the setup script](#)
- [6. Install MagicQ \(if not done by the script\)](#)
- [6.1 Qt5 and runtime dependencies](#)
- [7. Compact Wing & panel mode – how it works](#)
- [7.1 Start directly in the "Touch Compact" panel](#)
- [8. Autostart \(what the script creates\)](#)
- [9. No window decorations for MagicQ](#)
- [10. All ethernet interfaces on DHCP](#)
- [11. Automatic USB stick mounting](#)
- [12. Autologin \(optional, for "boot straight to MagicQ"\)](#)
- [13. Splashscreen \(hide boot logs\)](#)
- [13.1 Troubleshooting: "black screen instead of splash"](#)
- [13.2 Troubleshooting: "exec: /usr/bin/X: not found"](#)
- [13.3 Troubleshooting: MagicQ "cannot create the data folder"](#)
- [13.4 Hide the mouse cursor](#)
- [13.5 Monitor should always stay on \(no power saving\)](#)
- [14. Troubleshooting \(short list\)](#)
- [15. Compact command overview](#)
- [Project structure](#)

1. Requirements / Hardware

Part	Recommendation
Wyse 3040	Atom x5-Z8350, 2 GB RAM, 8 GB eMMC, UEFI-only (no legacy boot)
USB stick	≥ 4 GB, UEFI-bootable (recommended: prepared with Ventoy)
Display	Dual DisplayPort → use an active DP→HDMI adapter (DP++ often does not work reliably)

Part	Recommendation
Compact Wing	connect via USB (no driver needed)

Wyse 3040 features:

- Storage appears as `/dev/mmcblk0` (eMMC), *not* `/dev/sda`.
- Only 64-bit UEFI boot is possible.
- The eMMC device often has a name containing a special character (e.g. `MMC H8G4a\x92`),

which can confuse installers → see section 3.

2. Prepare the BIOS

1. Power on the device and press **F2** repeatedly.
2. If locked: unlock the BIOS with the password **Fireport** (or `Fireport2`).
3. **General → Boot Sequence** : put the USB stick first.
4. Note the **File Name** of the boot option (usually `\EFI\B00T\B00TX64.EFI`).
5. The Wyse later needs exactly this path on the internal eMMC (see section 4).
6. Optional: enable `Maintenance → Data Wipe → Wipe on Next Boot` to erase the eMMC

(if the same store has an old ThinOS installation).

1. Save and restart.

3. Install Ubuntu (recommended: 24.04 LTS Server)

Important: The newer Ubuntu installer (26.04) sometimes cannot find the eMMC / freezes.

Ubuntu 24.04 LTS is proven to work.

1. Put Ubuntu **22.04 or 24.04 LTS (ideally the Server image)** on the USB stick

(e.g. with Ventoy or `rufus / dd`).

1. Insert the USB stick, open the boot menu with **F12**, boot from the USB stick.
2. Choose the **Server installation** (no desktop environment is needed – we build

Openbox ourselves, which saves a lot of RAM/space on the small eMMC).

1. At the disk selection: choose the eMMC (`/dev/mmcblk0`). If it is **not shown**,

this is caused by the known device-name problem:

- Boot the live system, open a terminal and run:

```
```bash
```

```
sudo rm /dev/mmcblk0
```

```
'''
```

( `udev` re-creates the device cleanly afterwards.)

1. **Important – install GRUB to the UEFI removable-media location:**
2. Do not let GRUB install to the eMMC, but into the **EFI path**.
3. After installation, follow section 4 (BOOTX64.EFI), otherwise you get

"No bootable devices found".

1. Create the user under which MagicQ should run:

```
```bash
```

```
sudo adduser chamsys
```

```
'''
```

(This script and the MagicQ autostart use the user `chamsys`.)

1. After installation, boot from the USB stick (see section 4).

4. Repair the boot path (BOOTX64.EFI) – absolutely required!

The Wyse 3040 boots **only** from `\EFI\BOOT\BOOTX64.EFI`. Freshly installed Ubuntu/GRUB however provides `\EFI\debian\grubx64.efi` or similar. Without the fallback path → "No bootable devices found".

1. Boot with the live USB stick.
2. Mount the eMMC boot partition:

```
```bash
```

```
sudo blkid /dev/mmcblk0p1 # should show TYPE="vfat"
```

```
sudo mkdir -p /mnt/p1
```

```
sudo mount /dev/mmcblk0p1 /mnt/p1
```

```
'''
```

1. Create the fallback:

```
```bash
```

```
sudo mkdir -p /mnt/p1/EFI/BOOT
```

```
sudo cp /mnt/p1/EFI/ubuntu/grubx64.efi /mnt/p1/EFI/BOOT/BOOTX64.EFI
```

```
# or if "debian":
```

```
sudo cp /mnt/p1/EFI/debian/grubx64.efi /mnt/p1/EFI/BOOT/BOOTX64.EFI
```

```
sudo umount /mnt/p1
```

```
'''
```

1. Remove the USB stick and restart. Ubuntu should now boot from the eMMC.

5. Run the setup script

Important: The script must be run as **root** (`sudo`) because it installs system packages. MagicQ however runs under the user **chamsys** . The script therefore deliberately creates/uses all user files under `/home/chamsys` (not under `/root`).

⚠ **Prerequisite:** The user `chamsys` must exist before you start the script:

```
```bash
sudo adduser chamsys
```
```

The script `setup_magicq_wyse.sh` does the following automatically:

- Creates `/home/chamsys/.config/openbox/` **first** (before it is accessed).
- Installs Openbox (lightweight window manager), xinit, **xserver-xorg** (the actual

X server — without it `startx` fails with `exec: /usr/bin/X: not found`), X-/USB base tools.

- Installs **Qt5, xcb and runtime dependencies** (including fix for the error

"could not load QT plugin xcb").

- Installs MagicQ if a `.deb` is in the same folder, and checks the

libraries with `ldd` .

- Sets up the Openbox autostart for `chamsys` :
- X starts automatically,
- MagicQ opens in **fullscreen / panel mode for the Compact Wing**,
- MagicQ is displayed **without window decorations**.
- Sets **all ethernet interfaces to DHCP** (via netplan, `en*`).
- Sets up **autologin** (`tty1` → straight to Openbox + MagicQ, without password).
- Installs the **Plymouth splashscreen** (`splash.png`) and sets GRUB to `quiet splash`

with `GRUB_GFXMODE=1280x800` .

- Sets the resolution to **1280x800** (GRUB/splash **and** X11 via

`/etc/X11/xorg.conf.d/11-resolution.conf` → MagicQ runs fullscreen at 1280x800).

- Creates the required user/system files and sets the owner to `chamsys` .
- **Shuts down the show PC automatically** as soon as MagicQ is exited (QUIT soft button);

allows `chamsys` passwordless `shutdown / systemctl poweroff` (sudoers rule).

Run the script:

```
chmod +x setup_magicq_wyse.sh
sudo ./setup_magicq_wyse.sh                # without .deb -> MagicQ later manually
sudo ./setup_magicq_wyse.sh magicq_ubuntu*.deb  # with .deb in the same folder
```

6. Install MagicQ (if not done by the script)

1. English download page: <<https://www.chamsys.co.uk/mqdownload/>> →

download the **Ubuntu (64 bit)** `.deb` .

1. Install:

```
```bash
sudo dpkg -i magicq_ubuntu_*.deb
pull in dependencies if necessary:
sudo apt-get -f install
```
```

1. MagicQ is installed to `/opt/magicq/` . Start it manually for testing:

```
```bash
sudo -u chamsys /opt/magicq/runmagicq.sh
```
```

> **LibGL error** when starting? Then:

```
> ```bash
> sudo mv /opt/magicq/lib/libstdc.so.6 /opt/magicq/lib/libstdc.so.6~
> ```
```

> also possibly export `QT_AUTO_SCREEN_SCALE_FACTOR=0` in `runmagicq.sh` .

6.1 Qt5 and runtime dependencies

Good to know:

- MagicQ **bundles its own Qt5 libraries** under `/opt/magicq/lib`

(`bin/mqqt` uses them via `LD_LIBRARY_PATH`). For a plain start, **no** Qt5 system packages are therefore strictly required.

- `sudo dpkg -i + apt-get -f install` only pulls in the dependencies *declared*

in the package. Non-Qt runtime libraries (GLU, USB, PortAudio, FFmpeg, GStreamer, Alsa) are sometimes **not** declared there and can still be missing at startup.

The setup script therefore automatically installs **both** safety nets:

| Purpose | Packages |
|-----------------------------|--|
| Qt5 (fallback + multimedia) | <code>libqt5core5a libqt5gui5 libqt5widgets5 libqt5network5 libqt5opengl5 libqt5printsupport5 libqt5xml5 libqt5sql5 libqt5multimedia5 libqt5multimediawidgets5 libqt5svg5 libqt5qml5 libqt5quick5</code> |
| OpenGL / X11 | <code>libglu1-mesa libgl1 libglx-mesa0 libxext6 libxrender1</code> |
| USB (wings/DMX interfaces) | <code>libusb-1.0-0 libusb-0.1-4</code> |
| Audio / Video | <code>libportaudio2 libasound2* ffmpeg libgstreamer1.0-0 libgstreamer-plugins-base1.0-0 gstreamer1.0-plugins-base gstreamer1.0-plugins-good</code> |

| Purpose | Packages |
|----------------|---|
| Archive / Base | <code>libarchive13</code> <code>zlib1g</code> <code>libglib2.0-0</code> <code>libstdc++6</code> |

****ALSA package name (`libasound2*`):**** varies depending on the Ubuntu version!

- **Ubuntu 22.04:** `libasound2`
- **Ubuntu 24.04+:** `libasound2t64` (the old `libasound2` no longer exists there → *"libasound2 has no installation candidate"*).

The script selects the correct name automatically depending on the version. Also `libglu1-mesa` , `libgl1` etc. are partly named as `t64` variants on 24.04+; the script tries the appropriate selection and catches failures.

Error "could not load QT plugin xcb":

This error occurs when the Qt5-xcb platform plugin cannot find its X11 libraries.
The script therefore additionally installs all required `libxcb*` packages:

| Purpose | Packages |
|-------------------|--|
| xcb plugin (Qt5) | <code>libxcb-xinerama0</code> <code>libxcb-cursor0</code> <code>libxcb-keysyms1</code> <code>libxcb-image0</code> <code>libxcb-render-util0</code> <code>libxcb-icccm4</code> <code>libxcb-shape0</code> <code>libxcb-xfixes0</code> <code>libxcb-xkb1</code> <code>libxcb-xinput0</code> <code>libxcb-randr0</code> <code>libxcb-sync1</code> <code>libxcb-shm0</code> <code>libxcb1</code> |
| XKB / Fonts / EGL | <code>libxkbcommon-x11-0</code> <code>libxkbcommon0</code> <code>libfontconfig1</code> <code>libfreetype6</code> <code>libx11-xcb1</code> <code>libegl1</code> <code>libgl1</code> <code>libglx-mesa0</code> |

****GL/Mesa package names (`libgl1.*`): `libgl1-mesa-glx` no longer exists in Ubuntu 24.04+**** (removed since 23.10, had been a transitional package for a long time). It is replaced by `libgl1` **and** `libglx-mesa0` . The script therefore always uses `libgl1` + `libglx-mesa0` — this works on 22.04 and 24.04 alike.

In addition, the start script `/usr/local/bin/start_magicq.sh` forces the xcb platform:

```
export QT_QPA_PLATFORM=xcb
export QT_PLUGIN_PATH=/opt/magicq/plugins
```

Check / retrofit manually:

```
sudo apt-get install -y --no-install-recommends \
  libqt5gui5 libxcb-xinerama0 libxcb-cursor0 libxcb-keysyms1 libxcb-image0 \
  libxcb-render-util0 libxcb-icccm4 libxcb-shape0 libxcb-xfixes0 libxcb-xkb1 \
  libxcb-xinput0 libxcb-randr0 libxcb-sync1 libxcb-shm0 libxcb1 \
  libxkbcommon-x11-0 libxkbcommon0 libfontconfig1 libfreetype6
```

7. Compact Wing & panel mode – how it works

The autostart starts MagicQ with the **full panel mode** (imitating a Compact console layout) in **fullscreen**. The Compact Wing switches MagicQ into **unlocked mode** (full DMX output) as soon as it is connected via USB.

- **Switch panel mode:** In MagicQ → Setup → View Settings → Panels → **Full Panel**.
- **Check the wing:** Setup → View Settings → Ports → MagicQ Wings & Interfaces = Yes (auto DMX) .
- The wing needs **no separate driver** (only the newer Compact Wings).
- For an old PC/extra wing (FTDI): Setup → View Settings → Ports → FTDI + VCP driver .

Configuration note: "Full panel mode" and "fullscreen" are set by the script via the autostart command line and a one-time config file. If MagicQ does not start in the desired mode automatically on the first run, save the wanted settings once – MagicQ then remembers the state across restarts.

7.1 Start directly in the "Touch Compact" panel

For **touchscreen-only show operation**, MagicQ can start directly in the **"Touch Compact"** panel (or "Touch Compact Faders"). **There is no command-line/script parameter** for this – MagicQ only has a few CLI arguments (e.g. `wand` , remote IP, playback-mode shortcut), but no panel selection. The panel is a **console setting** that is set once in the GUI and then remembered by MagicQ across restarts – that is sufficient for the autostart, because our `start_magicq.sh` always loads the same show environment.

Set it up once on the Wyse (GUI):

1. Start MagicQ.
2. Setup → View Settings → Panels → select **Touch Compact** (or "Touch Compact Faders").
3. Save console settings: **SAVE SHOW** (or "Save Console Settings"), so the panel

is safely saved together with the loaded show environment.

1. Set Setup → View Settings → Windows → Start Mode to **None** – MagicQ then starts directly

into the saved environment instead of the "Choose demo show" dialog.

Afterwards MagicQ starts directly in the **Touch Compact** panel on every autostart.

Note: Since loading a *different* show by default loads only show data (without console settings), the panel remains in place as long as we always load the same show at boot (exactly what `start_magicq.sh` does).

8. Autostart (what the script creates)

- `~/ .config/openbox/autostart` – run at Openbox login and contains:

```
```bash
X server is started via .xinitrc / xinit (if not already running)
```

```
feh --bg-scale /usr/share/backgrounds/warty-final-ubuntu.png &
Mount USB sticks automatically (in addition to the udev rules)
for d in /dev/sd*; do [-b "$d"] && udisksctl mount -b "$d" 2>/dev/null; done &
Start MagicQ (fullscreen / Compact panel mode)
sleep 5
/usr/local/bin/start_magicq.sh &
````
```

- `/usr/local/bin/start_magicq.sh` – starts MagicQ with the correct config argument.
- **udev rule** `/etc/udev/rules.d/99-magicq.rules` (optional, created by the script) for

automatically mounting USB sticks.

Automatic shutdown: As soon as MagicQ is quit (e.g. via the **QUIT** soft button), `start_magicq.sh` automatically **shuts down** the show PC (`shutdown -h now`). This also happens if MagicQ crashes or exits with an error code – reliable for a show PC that otherwise is only operated via autostart without a keyboard.

9. No window decorations for MagicQ

So that MagicQ fills the entire screen **without title and window frames** (only the content), the script configures Openbox via `~/.config/openbox/rc.xml` . The inserted application rule fragment looks like this:

```
<applications>
  <application class="*/*MagicQ*">
    <decor>no</decor>          <!-- no window decorations -->
    <fullscreen>yes</fullscreen>
  </application>
</applications>
```

- `<decor>no</decor>` removes the window frame (no close/minimize button).
- `<fullscreen>yes</fullscreen>` ensures MagicQ uses the whole screen.

Edit manually afterwards:

```
nano ~/.config/openbox/rc.xml
# check/adjust the <applications> section, save
openbox --reconfigure
```

Note: The window class name may differ depending on the MagicQ version. If the rule does not take effect, check the class in an X terminal:

```
``bash
xprop | grep WM_CLASS # after focusing the MagicQ window
````
```

and adjust the value in the `class=` line accordingly.

---

## 10. All ethernet interfaces on DHCP

---

The script creates a netplan file `/etc/netplan/99-magicq-dhcp.yaml` that sets **all** ethernet interfaces (pattern `en*`) to DHCP (IPv4 **and** IPv6):

```
network:
 version: 2
 renderer: networkd
 ethernets:
 all-eth:
 match:
 name: "en*"
 dhcp4: yes
 dhcp6: yes
```

Activate once after running the script:

```
sudo netplan apply
ip a # check: en* interfaces have 192.168.x.x assigned
```

**Boot hangs without a network cable:** The script additionally **masks** the service `systemd-networkd-wait-online.service`. Without this mask, `network-online.target` waits **indefinitely** when there is no carrier (cable not plugged in) → boot stalls.

With the waiter masked, the Wyse boots straight through, even without a cable; as soon as a cable is plugged in, it still gets an IP via DHCP (networking runs in the background).

MagicQ does not need networking to boot.

```
sudo systemctl mask systemd-networkd-wait-online.service
```

The Wyse 3040 has a 1 Gbit ethernet port by default. Via a USB-ethernet adapter (for light show/Art-Net/sACN over a second network) this is likewise recognized as `en*` and configured via DHCP automatically. If you want to separate multiple networks (show network vs. Art-Net), you can add an interface with a static IP via `networkd` / `netplan` – DHCP remains active for the rest.

---

## 11. Automatic USB stick mounting

---

The script automatically chooses the appropriate method **depending on the Ubuntu version**:

- **Ubuntu 22.04 and older:** installs `usbmount` (simplest solution), which mounts

sticks automatically to `/media/usb*`.

- **Ubuntu 24.04 and newer:** `usbmount` no longer exists there – the command

`apt-get install usbmount` would fail with *"Unable to locate package usbmount"*.

The script skips `usbmount` on these versions and uses **udev + udisks** instead (see below).

**udev + udisks** (standard on modern systems / fallback):

The script creates the rule `/etc/udev/rules.d/99-magicq-usb.rules` and sets the owner.

In addition, the Openbox autostart mounts sticks already plugged in at boot:

```
for d in /dev/sd[b-z]*; do
 [-b "$d"] && udiskctl mount -b "$d" 2>/dev/null || true
done &
```

---

## 12. Autologin (optional, for "boot straight to MagicQ")

---

The setup script sets this up automatically: after power-on, the user is logged in on `tty1` without a password, `startx` is started and Openbox + MagicQ open in fullscreen – completely without interaction.

**What the script does for this:**

1. **systemd autologin** via an override file

```
/etc/systemd/system/getty@tty1.service.d/autologin.conf :
```

```
...
```

```
[Service]
```

```
ExecStart=
```

```
ExecStart=-/sbin/agetty --autologin chamsys --noclear tty1 linux
```

```
...
```

This logs the user **chamsys** in automatically on `tty1`.

1. `/home/chamsys/.bash_profile` is extended with a start block that runs `startx`

as soon as you land on `tty1`:

```
```bash
```

```
if [ -z "$DISPLAY" ] && [ "$(tty)" = "/dev/tty1" ]; then
```

```
exec startx
```

```
fi
```

```
```
```

1. `/home/chamsys/.xinitrc` starts `openbox-session` (from section 5), so that

after `startx` Openbox appears with the MagicQ autostart (fullscreen, no decorations).

All files belong to the user **chamsys** ( `chown chamsys:chamsys` ).

**Rebuild manually or switch to another user:**

```
sudo mkdir -p /etc/systemd/system/getty@tty1.service.d
sudo tee /etc/systemd/system/getty@tty1.service.d/autologin.conf >/dev/null <<'EOF'
[Service]
ExecStart=
ExecStart=-/sbin/agetty --autologin chamsys --noclear tty1 linux
EOF
sudo systemctl daemon-reload
sudo systemctl enable getty@tty1.service
```

**Disable (require the login password again):**

```
sudo rm /etc/systemd/system/getty@tty1.service.d/autologin.conf
sudo systemctl daemon-reload
```

---

## 13. Splashscreen (hide boot logs)

---

So that **no kernel/system logs**, but **your image splash.png** is shown fullscreen at boot, the script sets up **Plymouth** and sets GRUB to `quiet splash`.

⚠ **Important:** `splash.png` must be in the **same folder** as `setup_magicq_wyse.sh` when you run the script – it is then found and included automatically.

**How it works (robust approach):**

The script builds its **own Plymouth theme named magicq-splash** based on the **script** module. This shows your `splash.png` as a full image across the entire screen.

⚠ **Why script and not backgrounds ?**

On **Ubuntu 24.04 (noble)** the `backgrounds` plugin is **not available in any package** (the file list of `plymouth / plymouth-themes` only contains `script.so`, `text.so`, `tribar.so`, `details.so`, `fade-throbber.so`, `space-flares.so`). An earlier theme using `ModuleName=backgrounds` therefore produced the message *"plugin backgrounds.so is missing"* at boot. We use **script.so** instead, which lives in the **plymouth base package** (main, always present) – so the module is guaranteed to be there. `label-pango.so` is **not** needed.

Procedure:

1. Installs `plymouth`, `plymouth-themes`, `plymouth-label`, `plymouth-theme-script`, `plymouth-theme-spinner`, `plymouth-theme-ubuntu-logo`, `plymouth-theme-ubuntu-text` (as a safety net).

1. Creates `/usr/share/plymouth/themes/magicq-splash/` and copies `splash.png` → `background.png` (fallback: the `ubuntu-logo` logo).

1. Writes a `magicq-splash.plymouth` file with `ModuleName=script` **plus** a `magicq-splash.script` file (the `script` user code that scales the image across the full screen area).

1. **Activates the theme as default.** On Ubuntu 24.04 (Noble) the binary `plymouth-set-default-theme` is missing (Ubuntu bug **LP #1596220** — `/usr/bin` only contains `plymouth`), and `update-alternatives --set` often reports success incorrectly. The script therefore sets the alternatives link `/etc/alternatives/default.plymouth` **directly** to our theme and **verifies** the result via `readlink`.

1. Writes to `/etc/default/grub`:

```
...
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash loglevel=3 vt.global_cursor_default=0"
GRUB_GFXMODE=1280x800
GRUB_GFXPAYLOAD_LINUX=keep
GRUB_TERMINAL_OUTPUT=console
GRUB_TIMEOUT=0
...
```

- **quiet splash** (mandatory!) → enables the splash + no kernel messages.
- **loglevel=3** → only errors/warnings on the console (practically nothing).
- **GRUB\_GFXMODE=1280x800 / GRUB\_GFXPAYLOAD\_LINUX=keep** → graphics console + splash

run at **1280x800**, so the splash runs fullscreen (prevents switching to text-VGA).

- **GRUB\_TERMINAL\_OUTPUT=console** → prevents GRUB from forcing output onto a foreign terminal.

1. Runs `update-initramfs -u` and `update-grub` and **afterwards checks** via `lsinitramfs` whether theme + `script.so` made it into the initramfs. If something is missing, the theme is set again and rebuilt.

**Check activation / set afterwards (without `plymouth-set-default-theme`):**

```
readlink /etc/alternatives/default.plymouth # must point to magicq-splash

If not active yet, point it directly to our theme:
sudo ln -sf /usr/share/plymouth/themes/magicq-splash/magicq-splash.plymouth \
 /etc/alternatives/default.plymouth
sudo update-initramfs -u
sudo update-grub
```

**Replace the image later:**

```
sudo cp /new/path/splash.png /usr/share/plymouth/themes/magicq-splash/background.png
sudo update-initramfs -u
```

```
sudo update-grub
```

### Test / check the splash:

```
Plymouth running?
sudo plymouth --ping && echo "Plymouth is running"

Our theme baked in?
lsinitramfs /boot/initrd.img-$(uname -r) | grep -i magicq-splash

script module (from the plymouth base) present?
ls /usr/lib/x86_64-linux-gnu/plymouth/script.so

GRUB parameters correct?
cat /boot/grub/grub.cfg | grep -i "quiet splash"
```

## 13.1 Troubleshooting: "black screen instead of splash"

A black screen instead of the splash almost always has one of these causes – check them in this order:

1. **splash is missing in the kernel parameters** → without `splash` Plymouth does not start at all

(only an empty console = black).

```
```bash
sudo nano /etc/default/grub
# GRUB_CMDLINE_LINUX_DEFAULT="quiet splash ..." <- splash MUST be in there
sudo update-grub
```
```

1. **Script module missing** → if a `ModuleName=script` theme is used and

`plymouth-theme-script` is not installed, `script.so` is missing and the theme will not load.

```
```bash
sudo apt-get install -y plymouth-theme-script
sudo update-initramfs -u
```
```

*(The script now installs this package automatically.)*

1. **label module missing / not in the initramfs** → "the plugin label-pango.so is missing".

The `label` module is in the `plymouth-label` package (or `plymouth-themes`).

**Important:** Installing the package is not enough – the `.so` file must additionally be embedded into the **initramfs** (this is the most common reason the error keeps occurring despite installation).

```
```bash
sudo apt-get install -y plymouth-themes plymouth-label
sudo update-initramfs -u # MUST run after the installation
# Check whether it was embedded:
```

```
lsinitramfs /boot/initrd.img-$(uname -r) | grep -i "label-pango\|label.so"
...

```

(The script avoids this problem from the start: it uses its own `script` theme that needs no `label` module at all.)

1. **plymouth-set-default-theme: command not found** → on **Ubuntu 24.04 (Noble)** this

binary is known to be missing from the `plymouth` package (Ubuntu bug **LP #1596220**); `/usr/bin` then only contains `plymouth`. This is not an installation error. In that case set the default theme via `update-alternatives`:

```
```bash
sudo update-alternatives --config default.plymouth
sudo update-initramfs -u
...

```

\*(The script detects this and activates the theme automatically via `update-alternatives --set default.plymouth` or symlink – it no longer relies on the missing command.)\*

1. **Theme missing in the initramfs** → `update-initramfs -u` was forgotten or failed.

```
```bash
sudo update-initramfs -u
lsinitramfs /boot/initrd.img-$(uname -r) | grep -i "ubuntu-logo\|magicq"
...

```

1. **Graphics driver/ nomodeset** → If `nomodeset` is in the kernel parameters,

Plymouth is often not shown. Remove `nomodeset` (unless it is strictly required for other reasons).

1. **Wrong resolution** → The script sets `GRUB_GFXMODE=1280x800` (boot/splash) and

additionally forces 1280x800 in X11 via `/etc/X11/xorg.conf.d/11-resolution.conf`. If the monitor has a different native resolution, adjust both values.

1. **Display hardware** → On the Wyse 3040 an **active DP→HDMI adapter** is required. Without the right adapter the screen may remain black.

1. **Splash flickers / repeatedly goes black** → Known Plymouth- `script` bug: the

framebuffer is cleared on every refresh cycle, and if the theme callback does not re-set the image **every time**, the screen repeatedly goes black. The script now redraws the image on every refresh (see `magicq-splash.script : draw_bg()` with `SetPosition` / `SetScale` / `SetOpacity` / `SetZ(15)` in the `refresh_callback`). After changes:

```
```bash
sudo update-initramfs -u && sudo reboot
...

```

\*(A short black flash **exactly once** shortly before the X desktop is, in contrast, the normal Plymouth → display manager transition and not a bug.)\*

1. **An (Ubuntu/vendor) logo appears instead of splash.png** →

`/etc/alternatives/default.plymouth` still points to another theme (e.g. `bgrt` or `ubuntu-logo`). Activating via `update-alternatives --set` often does not take effect on Noble. Point directly to our theme and rebuild:

```
```bash
readlink /etc/alternatives/default.plymouth
sudo ln -sf /usr/share/plymouth/themes/magicq-splash/magicq-splash.plymouth \
/etc/alternatives/default.plymouth
sudo update-initramfs -u && sudo update-grub
sudo reboot
```
```

### Show boot logs at all (only for diagnostics):

Hold **Shift** (or `Esc` in GRUB) during boot to open the GRUB menu and remove the `quiet splash` kernel parameters via `e` – then all messages appear again.

## 13.2 Troubleshooting: "exec: /usr/bin/X: not found"

---

This error appears on autologin/ `startx` from `/etc/X11/xinit/xserverrc` and means that **the X server itself is not installed**. `xinit` / `startx` need `xserver-xorg`, which provides the `/usr/bin/X` start path.

Fix:

```
sudo apt-get install -y xserver-xorg
sudo update-alternatives --install /usr/bin/X x-session-manager /usr/bin/Xorg 50
startx
```

Check:

```
ls -la /usr/bin/X # must point to /usr/bin/Xorg
which Xorg
```

*(The script now installs `xserver-xorg` automatically in step 1.)*

## 13.3 Troubleshooting: MagicQ "cannot create the data folder"

---

MagicQ stores show files and settings in `~/MagicQ` or under `~/.config/MagicQ` and `~/.local/share/Chamsys`. If these are missing or not writable, MagicQ reports it cannot create the data folder.

Manual fix (as root):

```
sudo mkdir -p /home/chamsys/MagicQ
sudo mkdir -p /home/chamsys/.config/MagicQ
sudo mkdir -p /home/chamsys/.local/share/Chamsys
```

```
sudo chown -R chamsys:chamsys /home/chamsys
ls -ld /home/chamsys/MagicQ # must belong to chamsys
```

\*(The script creates the data folder in step 5e automatically and forces `$HOME` in the start script, so MagicQ finds the right location.)\*

## 13.4 Hide the mouse cursor

No mouse pointer should be visible on the show PC (neither in the Openbox desktop nor in the Qt application MagicQ). The script generates a **valid transparent XCursor theme** in **XCursor binary format** and places it **system-wide**:

1. Builds the theme `Transparent` in `/usr/share/icons/Transparent` (always found, independent of `$HOME`) and mirrors it additionally into `~/.icons/Transparent`.

1. Sets `/usr/share/icons/default` → `Transparent` (the **global X default**):

Even when Qt/MagicQ requests a cursor name, Xcursor resolves it system-wide against our transparent theme – this is the decisive fix for the pointer **\*\*over the MagicQ window\*\*** (previously the theme only lived in the home directory and was not found when `$HOME` /the search path differed).

1. Sets `XCURSOR_THEME` / `XCURSOR_SIZE` globally in `/etc/environment` and sets

`gtk-cursor-theme-name` in `~/.config/gtk-3.0/settings.ini`, so Qt **and** GTK use the theme.

**⚠ Why a pure binary format is needed:** The earliest version placed **raw PNG files** under `cursors/` – but Xcursor requires the **XCursor file format** (`Xcur` header) and silently ignored the PNGs (the arrow stayed visible). The script writes the `.cursor` file **directly via Python** (exactly matching `XcursorXcFileSave` from `libXcursor`: magic `0x72756358`, version `0x00010000`, one 32×32 image chunk with `type=0xfffd0002`, fully transparent ARGB pixels) – completely **without** external tools. `xcursorgen` (from `x11-apps`) only serves as a fallback if `python3` is missing. This guarantees the files are **>0 bytes** and loadable by Xcursor **and** Qt.

Additionally:

- the script sets `xsetroot -cursor_name none` in `~/.xinitrc` **before** `exec openbox-session`

(root pointer invisible) and again `XCURSOR_THEME` / `XCURSOR_SIZE` + `xsetroot` in `/usr/local/bin/start_magicq.sh`, so the transparent pointer takes effect directly in the MagicQ process.

`x11-apps` (provides `xcursorgen`) is in the universe repo; it is only needed as a **fallback** – the Python writer works without it. Check manually:

```
ls -l /usr/share/icons/Transparent/cursors/default # MUST be >0 bytes (4160 bytes for 32×32)
xxd /usr/share/icons/Transparent/cursors/default | head -1 # starts with: 5863 7572 ("Xcur")
cat /usr/share/icons/default/index.theme # Inherits=Transparent
cat /etc/environment # XCURSOR_THEME=Transparent
```

## 13.5 Monitor should always stay on (no power saving)

To ensure the screen on the show PC **never** goes into standby/save mode, the script disables this in the X start (Openbox autostart **and** in `start_magicq.sh`):

```
xset s off # X screensaver off
xset -dpms # DPMS (standby/suspend/off) completely off
xset dpms 0 0 0 # timeouts to "never"
setterm -blank 0 -powersave off -powerdown 0 # never blank the text console
```

Manually on the box (in the running X):

```
xset s off
xset -dpms
xset dpms 0 0 0
```

If the monitor still powers off, additionally check the monitor's own settings (auto standby) and whether the amplifier/DP adapter is controlling it.

## 14. Troubleshooting (short list)

| Problem                                  | Solution                                                                                                                                           |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| "No bootable devices found"              | Follow section 4 (BOOTX64.EFI)                                                                                                                     |
| Installer cannot find eMMC               | <code>sudo rm /dev/mmcblk0</code> in the live system                                                                                               |
| Installer freezes                        | Avoid newer Ubuntu versions; Go to 24.04 LTS                                                                                                       |
| MagicQ LibGL error                       | Rename <code>libstdc.so.6</code> (section 6)                                                                                                       |
| UI too small/distorted                   | <code>QT_AUTO_SCREEN_SCALE_FACTOR=0</code> in <code>runmagicq.sh</code>                                                                            |
| Wing not detected                        | Ports → MagicQ Wings & Interfaces = Yes (auto DMX)                                                                                                 |
| No image on HDMI                         | Use an active DP→HDMI adapter                                                                                                                      |
| MagicQ needs root for USB                | Script starts MagicQ with sudo if needed / sets an appropriate udev rule                                                                           |
| MagicQ still has window frames           | Check <code>~/.config/openbox/rc.xml</code> + <code>openbox --reconfigure</code> (section 9)                                                       |
| <code>exec: /usr/bin/X: not found</code> | <code>sudo apt-get install xserver-xorg</code> (section 13.2)                                                                                      |
| MagicQ: "cannot create the data folder"  | <code>~/.config/MagicQ</code> / <code>~/MagicQ</code> missing/not writable; create with <code>chown -R chamsys:chamsys + mkdir</code> (section 5e) |

| Problem                                         | Solution                                                                                                                                                                                                                                                                                                                          |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| XML syntax error in<br>~/.config/openbox/rc.xml | rc.xml broken/empty → the script now rewrites it completely; manually<br><code>sudo apt-get install --reinstall openbox</code> or delete the file so it is recreated                                                                                                                                                              |
| Ethernet gets no IP                             | <code>sudo netplan apply</code> + check <code>ip a</code> (section 10)                                                                                                                                                                                                                                                            |
| Boot logs appear despite splash                 | Check GRUB <code>quiet splash</code> (section 13)                                                                                                                                                                                                                                                                                 |
| Ubuntu logo instead of our splash               | Link <code>/etc/alternatives/default.plymouth</code> directly to <code>magicq-splash</code> + <code>update-initramfs</code> (section 13.1 point 8; often the <code>bgrt</code> theme is the cause)                                                                                                                                |
| Monitor goes into standby/save mode             | <code>xset -dpms</code> , <code>xset s off</code> , <code>xset dpms 0 0 0</code> in the X start (section 13.5)                                                                                                                                                                                                                    |
| Mouse cursor visible (should be invisible)      | Script writes a valid transparent theme in XCursor binary format directly via Python ( <code>Xcur</code> , 4 KB file) system-wide +<br><code>/usr/share/icons/default</code> → <code>Transparent</code> (section 13.4). Check: <code>ls -l /usr/share/icons/Transparent/cursors/left_ptr</code> must be <b>&gt;0 bytes</b> (4160) |
| Black screen instead of splash                  | Section 13.1 (script module, <code>splash</code> parameter, <code>nomodeset</code> , resolution, DP adapter)                                                                                                                                                                                                                      |
| Missing shared library / Qt5 error              | <code>cd /opt/magicq &amp;&amp; LD_LIBRARY_PATH=/opt/magicq/lib ldd ./bin/mqqt</code> + <code>sudo apt-get -f install</code> (section 6.1)                                                                                                                                                                                        |
| "could not load QT plugin xcb"                  | Install <code>libxcb-*</code> packages, set <code>QT_QPA_PLATFORM=xcb</code> (section 6.1)                                                                                                                                                                                                                                        |
| No image / black screen after boot              | Use an active DP→HDMI adapter, check <code>QT_SCREEN_SCALE_FACTORS=1</code>                                                                                                                                                                                                                                                       |

## 15. Compact command overview

```
Start MagicQ manually
/opt/magicq/runmagicq.sh

Edit the autostart file
nano ~/.config/openbox/autostart

Test USB mount
udisksctl mount -b /dev/sdb1
ls /media/ # mount point for usbmount

Reload Openbox config (window decorations)
```

```

openbox --reconfigure
nano ~/.config/openbox/rc.xml

Enable DHCP for all ethernet interfaces
sudo netplan apply
ip a

Disable autologin (ask for password again)
sudo rm /etc/systemd/system/getty@tty1.service.d/autologin.conf
sudo systemctl daemon-reload

Re-apply splashscreen theme / update GRUB
(on Ubuntu 24.04 there is no plymouth-set-default-theme; set the link directly)
sudo ln -sf /usr/share/plymouth/themes/magicq-splash/magicq-splash.plymouth \
 /etc/alternatives/default.plymouth
sudo update-initramfs -u
sudo update-grub

Make the mouse cursor invisible (no-package fallback; theme is created by the script in the home
sudo mkdir -p /usr/share/icons/default
sudo sh -c 'echo -e "[Icon Theme]\nInherits=Transparent" > /usr/share/icons/default/index.theme'
echo 'xsetroot -cursor_name none' >> ~/.xinitrc

Monitor should never go into save mode/standby (in the X server)
xset s off
xset -dpms
xset dpms 0 0 0

Restart
sudo reboot

```

## Project structure

---

```

Linux-Installer/
├─ README.md # This handbook
├─ setup_magicq_wyse.sh # Setup script
└─ splash.png # Plymouth splash screen

```